

Real-Time Examples of the Factory Design Pattern in C#

In this article, I will discuss **Multiple Real-Time Examples of the Factory Design Pattern in C#**. Please read our previous article discussing the basic concepts of the [Factory Design Pattern in C#](#) with Examples. At the end of this article, you will understand the following Real-time Examples using the Factory Design Pattern in C#.

1. [Payment Gateway Integration](#)
2. [Document Conversion System](#)
3. [Logging System](#)
4. [A Simple System to Handle Notifications](#)
5. [Discounts in an E-commerce Application](#)
6. [Transport Application](#)
7. [Developing a Graphics Editor](#)
8. [Designing a System for a Bank](#)
9. [Report Generation](#)
10. [Cloud Storage System](#)
11. [UI Theme System](#)

Real-Time Example of Factory Design Pattern in C#: Payment Gateway Integration

Suppose you're building an e-commerce system that allows multiple payment gateways like PayPal, Stripe, and Credit Card processing. The system will use the appropriate payment gateway depending on the user's selection or regional restrictions. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```
using System;
namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface IPaymentGateway
    {
        void ProcessPayment(decimal amount);
    }
    //Concrete Implementations for the Products
    public class PayPalGateway : IPaymentGateway
    {
        public void ProcessPayment(decimal amount)
        {
            Console.WriteLine($"Processing ${amount} payment through
PayPal...");
            // Actual integration and logic for PayPal
        }
    }
    public class StripeGateway : IPaymentGateway
    {
        public void ProcessPayment(decimal amount)
        {
            Console.WriteLine($"Processing ${amount} payment through
Stripe...");
            // Actual integration and logic for Stripe
        }
    }
    public class CreditCardGateway : IPaymentGateway
    {
        public void ProcessPayment(decimal amount)
```

```

        {
            Console.WriteLine($"Processing ${amount} payment using Credit
Card...");
            // Logic for direct credit card processing
        }
    }
    //Factory Class to Produce the Products
    public static class PaymentGatewayFactory
    {
        public static IPaymentGateway CreatePaymentGateway(string
gatewayName)
        {
            switch (gatewayName.ToLower())
            {
                case "paypal":
                    return new PayPalGateway();
                case "stripe":
                    return new StripeGateway();
                case "creditcard":
                    return new CreditCardGateway();
                default:
                    throw new ArgumentException("Invalid payment gateway
specified");
            }
        }
    }

    // Testing the Factory Design Pattern
    public class Program
    {
        public static void Main()
        {
            Console.WriteLine("Select the payment gateway (PayPal, Stripe,
CreditCard): ");
            string gatewayName = Console.ReadLine();
            try
            {
                IPaymentGateway paymentGateway =
PaymentGatewayFactory.CreatePaymentGateway(gatewayName);
                paymentGateway.ProcessPayment(100.00M); // Example amount
            }
            catch (ArgumentException ex)
            {
                Console.WriteLine(ex.Message);
            }
            Console.ReadKey();
        }
    }
}

```

Expanding the e-commerce system to include new payment methods becomes easy with this design. Suppose a new payment method, like Bitcoin, is introduced in the future. In that case, you only need to implement a new BitcoinGateway class and extend the PaymentGatewayFactory without altering the existing client code. This encapsulation of the creation process makes the Factory Design Pattern a powerful tool for such scenarios, ensuring scalability and maintainability. When you run the above code, you will get the following output.

```
Select the payment gateway (PayPal, Stripe, CreditCard):
Paypal
Processing $100.00 payment through PayPal...
```

Real-Time Example of Factory Design Pattern in C#: Document Conversion System

Imagine a software system that allows users to convert documents between different formats like DOCX, PDF, and TXT. The system will utilize the appropriate converter depending on the user's request. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```
using System;
namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface IDocumentConverter
    {
        string Convert(string content);
        string TargetExtension { get; }
    }
    //Concrete Implementations for the Products
    public class DocxConverter : IDocumentConverter
    {
        public string Convert(string content)
        {
            Console.WriteLine("Converting content to DOCX format...");
            // Logic for DOCX conversion, simplified for this example
            return content + " [Converted to DOCX]";
        }
        public string TargetExtension => ".docx";
    }
    public class PdfConverter : IDocumentConverter
    {
        public string Convert(string content)
        {
            Console.WriteLine("Converting content to PDF format...");
            // Logic for PDF conversion, simplified for this example
            return content + " [Converted to PDF]";
        }
        public string TargetExtension => ".pdf";
    }
    public class TxtConverter : IDocumentConverter
    {
        public string Convert(string content)
        {
            Console.WriteLine("Converting content to TXT format...");
            // Logic for TXT conversion, simplified for this example
            return content + " [Converted to TXT]";
        }
        public string TargetExtension => ".txt";
    }
    //Factory Class to Produce the Products
    public static class DocumentConverterFactory
    {
```

```

        public static IDocumentConverter CreateDocumentConverter(string
format)
    {
        switch (format.ToLower())
        {
            case "docx":
                return new DocxConverter();
            case "pdf":
                return new PdfConverter();
            case "txt":
                return new TxtConverter();
            default:
                throw new ArgumentException("Unsupported document
format");
        }
    }
}

// Testing the Factory Design Pattern
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Enter the content to convert:");
        string content = Console.ReadLine();
        Console.WriteLine("Select the target format (DOCX, PDF,
TXT):");

        string format = Console.ReadLine();
        try
        {
            IDocumentConverter converter =
DocumentConverterFactory.CreateDocumentConverter(format);
            string convertedContent = converter.Convert(content);
            Console.WriteLine($"Converted content
({converter.TargetExtension}): {convertedContent}");
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine(ex.Message);
        }
        Console.ReadKey();
    }
}

```

This design enables the system to accommodate new document formats with ease. If a new format, say RTF, is introduced later, one would only have to implement a new `RtfConverter` class and update the `DocumentConverterFactory` without changing the core logic. The Factory Design Pattern ensures modularity and separation of concerns, making expanding or modifying functionalities easier. When you run the above code, you will get the following output.

```
Enter the content to convert:
Welcome to Dot Net Tutorials by Pranaya Rout
Select the target format (DOCX, PDF, TXT):
PDF
Converting content to PDF format...
Converted content (.pdf): Welcome to Dot Net Tutorials by Pranaya Rout [Converted to PDF]
```

Real-Time Example of Factory Design Pattern in C#: Logging System

Let's consider a real-time example of a logging system. Many applications must log messages to destinations like consoles, files, or remote servers. Using the Factory Design Pattern, we can decouple the client code from specific logger implementations and easily switch between different logging strategies. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```
using System;
using System.IO;
namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface ILogger
    {
        void Log(string message);
    }
    //Concrete Implementations for the Products
    public class ConsoleLogger : ILogger
    {
        public void Log(string message)
        {
            Console.WriteLine("Console: " + message);
        }
    }
    public class FileLogger : ILogger
    {
        private string _filePath;
        public FileLogger(string filePath)
        {
            _filePath = filePath;
        }
        public void Log(string message)
        {
            File.AppendAllText(_filePath, "File: " + message +
Environment.NewLine);
        }
    }
    // You can also add other loggers like DatabaseLogger, CloudLogger,
    etc.
    //Factory Class to Produce the Products
    public static class LoggerFactory
    {
        public static ILogger CreateLogger(string type)
        {
            switch (type.ToLower())
            {
                case "console":
                    return new ConsoleLogger();
            }
        }
    }
}
```

```

        case "file":
            return new FileLogger("log.txt");
            // For simplicity, file path is hardcoded
            // You can extend here with other logger types
        default:
            throw new ArgumentException("Invalid logger type");
    }
}

// Testing the Factory Design Pattern
public class Program
{
    public static void Main()
    {
        ILogger logger;
        logger = LoggerFactory.CreateLogger("console");
        logger.Log("This is a console log!");
        logger = LoggerFactory.CreateLogger("file");
        logger.Log("This message is written to a file.");
        // The beauty of this approach is that the client code remains
        // even if we introduce new logger types in the future.
        Console.ReadKey();
    }
}

```

unchanged

In this example, we've utilized the Factory Design Pattern to easily switch between logging strategies without changing the client code. Suppose in the future, a new requirement emerges, like logging into a database. In that case, we can extend our LoggerFactory to handle the new logger type, keeping the client code and existing logger implementations untouched.

Real-Time Example of Factory Design Pattern in C#: A Simple System to Handle Notifications

Let's understand a different real-time scenario: a simple system to handle notifications. In this scenario, an application may need to send different types of notifications to users, such as email notifications, SMS notifications, or push notifications. Using the Factory Design Pattern, we can easily produce the desired notification sender based on the user's preferences or the context. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```

using System;
namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface INotificationSender
    {
        void SendNotification(string message);
    }
    //Concrete Implementations for the Products
    public class EmailNotification : INotificationSender
    {
        public void SendNotification(string message)
        {

```

```

        Console.WriteLine($"Sending EMAIL notification: {message}");
        // Logic to send email here...
    }
}
public class SMSNotification : INotificationSender
{
    public void SendNotification(string message)
    {
        Console.WriteLine($"Sending SMS notification: {message}");
        // Logic to send SMS here...
    }
}
public class PushNotification : INotificationSender
{
    public void SendNotification(string message)
    {
        Console.WriteLine($"Sending PUSH notification: {message}");
        // Logic to send push notification here...
    }
}
//Factory Class to Produce the Products
public static class NotificationFactory
{
    public static INotificationSender CreateNotificationSender(string
type)
    {
        switch (type.ToLower())
        {
            case "email":
                return new EmailNotification();
            case "sms":
                return new SMSNotification();
            case "push":
                return new PushNotification();
            default:
                throw new ArgumentException("Invalid notification
type");
        }
    }
}

// Testing the Factory Design Pattern
public class Program
{
    public static void Main()
    {
        INotificationSender notificationSender;
        notificationSender =
NotificationFactory.CreateNotificationSender("email");
        notificationSender.SendNotification("This is an email
notification!");
        notificationSender =
NotificationFactory.CreateNotificationSender("sms");

```

```

        notificationSender.SendNotification("This is an SMS
notification!");
        notificationSender =
NotificationFactory.CreateNotificationSender("push");
        notificationSender.SendNotification("This is a push
notification!");
        // As with other factory examples, adding new notification
methods
        // would only require extending the factory, without altering
the client code.
        Console.ReadKey();
    }
}
}

```

In this real-time example, the Factory Design Pattern enables us to abstract away the details of creating different notification senders. As a result, if we need to add more notification types in the future, we can easily expand our factory while the client code remains consistent and unaltered. When you run the above code, you will get the following output.

```

Sending EMAIL notification: This is an email notification!
Sending SMS notification: This is an SMS notification!
Sending PUSH notification: This is a push notification!

```

Real-Time Example of Factory Design Pattern in C#: Discounts in an E-commerce Application

Let's understand another real-time example where the Factory Design Pattern can be applied. Consider an e-commerce platform where different types of discounts are applied to products based on different criteria: seasonal offers, clearance sales, and member discounts.

Using the Factory Design Pattern, we can create a discount strategy for a product without the client code knowing the specifics of which discount strategy to apply. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```

using System;
namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface IDiscountStrategy
    {
        decimal ApplyDiscount(decimal price);
    }
    //Concrete Implementations for the Products
    public class SeasonalDiscount : IDiscountStrategy
    {
        public decimal ApplyDiscount(decimal price)
        {
            return price * 0.90m; // 10% seasonal discount
        }
    }
    public class ClearanceDiscount : IDiscountStrategy
    {
        public decimal ApplyDiscount(decimal price)
    }
}

```

```

        {
            return price * 0.70m; // 30% clearance discount
        }
    }
    public class MemberDiscount : IDiscountStrategy
    {
        public decimal ApplyDiscount(decimal price)
        {
            return price * 0.95m; // 5% member discount
        }
    }
    //Factory Class to Produce the Products
    public static class DiscountFactory
    {
        public static IDiscountStrategy CreateDiscountStrategy(string type)
        {
            switch (type.ToLower())
            {
                case "seasonal":
                    return new SeasonalDiscount();
                case "clearance":
                    return new ClearanceDiscount();
                case "member":
                    return new MemberDiscount();
                default:
                    throw new ArgumentException("Invalid discount type");
            }
        }
    }
}

// Testing the Factory Design Pattern
public class Program
{
    public static void Main()
    {
        IDiscountStrategy discountStrategy;
        decimal originalPrice = 100.0m;
        discountStrategy =
DiscountFactory.CreateDiscountStrategy("seasonal");
        Console.WriteLine($"Seasonal Discounted Price:
{discountStrategy.ApplyDiscount(originalPrice)}");
        discountStrategy =
DiscountFactory.CreateDiscountStrategy("clearance");
        Console.WriteLine($"Clearance Discounted Price:
{discountStrategy.ApplyDiscount(originalPrice)}");
        discountStrategy =
DiscountFactory.CreateDiscountStrategy("member");
        Console.WriteLine($"Member Discounted Price:
{discountStrategy.ApplyDiscount(originalPrice)}");
        Console.ReadKey();
    }
}
}

```

With this setup, whenever a new type of discount is added, we must extend the DiscountFactory and create a new discount strategy class. The client code remains consistent and doesn't need to be aware of the specific details about how each discount is applied. This demonstrates how the Factory Design Pattern allows for a clear separation of concerns and offers flexibility in object creation. When you run the above code, you will get the following output.

```
Seasonal Discounted Price: 90.000
Clearance Discounted Price: 70.000
Member Discounted Price: 95.000
```

Real-Time Example of Factory Design Pattern in C#: Transport Application

Let's consider another real-world example. Imagine an application scenario where users can choose between different transportation modes, like a car, bus, or bike. Using the Factory Design Pattern, the application can instantiate the desired transportation mode based on user choice without directly coupling to specific transportation classes. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```
using System;
namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface ITransport
    {
        void StartRoute();
    }
    //Concrete Implementations for the Products
    public class Car : ITransport
    {
        public void StartRoute()
        {
            Console.WriteLine("Starting the journey with a car!");
            // Logic related to starting a car route
        }
    }
    public class Bus : ITransport
    {
        public void StartRoute()
        {
            Console.WriteLine("Starting the journey with a bus!");
            // Logic related to starting a bus route
        }
    }
    public class Bike : ITransport
    {
        public void StartRoute()
        {
            Console.WriteLine("Starting the journey with a bike!");
            // Logic related to starting a bike route
        }
    }
    //Factory Class to Produce the Products
    public static class TransportFactory
    {
        public static ITransport CreateTransport(string type)
        {

```

```

        switch (type.ToLower())
        {
            case "car":
                return new Car();
            case "bus":
                return new Bus();
            case "bike":
                return new Bike();
            default:
                throw new ArgumentException("Invalid transport type");
        }
    }
}

// Testing the Factory Design Pattern
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Please enter the mode of transport (car, bus, bike): ");
        string userChoice = Console.ReadLine();
        try
        {
            ITransport chosenTransport =
TransportFactory.CreateTransport(userChoice);
            chosenTransport.StartRoute();
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine(ex.Message);
        }
        Console.ReadKey();
    }
}

```

In this real-time example, the Factory Design Pattern allows us to decouple the decision-making process from the client code, making it easier to extend the program with new modes of transportation in the future. If, for instance, you wanted to introduce a “train” mode, you’d need to add a new Train class and update the TransportFactory without altering the existing client code. When you run the above code, you will get the following output.

```

Please enter the mode of transport (car, bus, bike):
car
Starting the journey with a car!

```

Real-Time Example of Factory Design Pattern in C#: Developing a Graphics Editor

Let’s explore a different scenario. Imagine you’re developing a graphics editor where users can draw various shapes. The editor should instantiate and draw the desired shape based on the user’s choice. We can encapsulate the logic for creating specific shape objects using the Factory Design Pattern. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```

using System;

```

```

namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface IShape
    {
        void Draw();
    }
    //Concrete Implementations for the Products
    public class Circle : IShape
    {
        public void Draw()
        {
            Console.WriteLine("Drawing a Circle!");
        }
    }
    public class Rectangle : IShape
    {
        public void Draw()
        {
            Console.WriteLine("Drawing a Rectangle!");
        }
    }
    public class Triangle : IShape
    {
        public void Draw()
        {
            Console.WriteLine("Drawing a Triangle!");
        }
    }
    //Factory Class to Produce the Products
    public static class ShapeFactory
    {
        public static IShape CreateShape(string shapeType)
        {
            switch (shapeType.ToLower())
            {
                case "circle":
                    return new Circle();
                case "rectangle":
                    return new Rectangle();
                case "triangle":
                    return new Triangle();
                default:
                    throw new ArgumentException("Invalid shape type");
            }
        }
    }

    // Testing the Factory Design Pattern
    public class Program
    {
        public static void Main()
        {
    
```

```

        Console.WriteLine("Which shape would you like to draw? (circle, rectangle, triangle): ");
        string userChoice = Console.ReadLine();
        try
        {
            IShape chosenShape = ShapeFactory.CreateShape(userChoice);
            chosenShape.Draw();
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine(ex.Message);
        }
        Console.ReadKey();
    }
}

```

The Factory Design Pattern helps maintain this example's Open/Closed Principle. If we wish to introduce new shapes in the future (e.g., Polygon, Ellipse), we would only need to extend the ShapeFactory and add the new shape class without modifying the existing client code. This approach provides a structured way to manage shape object creation, offering flexibility and encapsulation. When you run the above code, you will get the following output.

```

Which shape would you like to draw? (circle, rectangle, triangle):
circle
Drawing a Circle!

```

Real-Time Example of Factory Design Pattern in C#: Designing a System for a Bank

Let's look into another real-world scenario. Suppose you're designing a bank system that supports various accounts: Savings, Current, and Fixed Deposit. Using the Factory Design Pattern, we can create a specific account type based on the user's choice or requirements. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```

using System;
namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface IAccount
    {
        decimal Balance { get; }
        void Deposit(decimal amount);
        void Withdraw(decimal amount);
    }
    //Concrete Implementations for the Products
    public class SavingsAccount : IAccount
    {
        public decimal Balance { get; private set; }
        public void Deposit(decimal amount)
        {
            Balance += amount;
            Console.WriteLine($"Deposited {amount} to Savings Account. New Balance: {Balance}");
        }
    }
}

```

```

        public void Withdraw(decimal amount)
        {
            // Implement logic specific to Savings account (e.g., limited
withdrawals, minimum balance requirement)
            Balance -= amount;
            Console.WriteLine($"Withdrew {amount} from Savings Account. New
Balance: {Balance}");
        }
    }
    public class CurrentAccount : IAccount
    {
        public decimal Balance { get; private set; }
        public void Deposit(decimal amount)
        {
            Balance += amount;
            Console.WriteLine($"Deposited {amount} to Current Account. New
Balance: {Balance}");
        }
        public void Withdraw(decimal amount)
        {
            // Implement logic specific to Current account (e.g., no
withdrawal limit)
            Balance -= amount;
            Console.WriteLine($"Withdrew {amount} from Current Account. New
Balance: {Balance}");
        }
    }
    public class FixedDepositAccount : IAccount
    {
        public decimal Balance { get; private set; }
        public void Deposit(decimal amount)
        {
            // Typically, you can't deposit into a fixed deposit account
once it's created. For simplicity, we'll allow it here.
            Balance += amount;
            Console.WriteLine($"Deposited {amount} to Fixed Deposit
Account. New Balance: {Balance}");
        }
        public void Withdraw(decimal amount)
        {
            // Implement logic specific to Fixed Deposit account (e.g.,
penalties for early withdrawal)
            Balance -= amount;
            Console.WriteLine($"Withdrew {amount} from Fixed Deposit
Account with penalty. New Balance: {Balance}");
        }
    }
    //Factory Class to Produce the Products
    public static class AccountFactory
    {
        public static IAccount CreateAccount(string accountType)
        {
            switch (accountType.ToLower())
            {

```

```

        case "savings":
            return new SavingsAccount();
        case "current":
            return new CurrentAccount();
        case "fixeddeposit":
            return new FixedDepositAccount();
        default:
            throw new ArgumentException("Invalid account type");
    }
}

// Testing the Factory Design Pattern
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Enter the type of account you'd like to
create (savings, current, fixeddeposit): ");
        string accountType = Console.ReadLine();
        try
        {
            IAccount account =
AccountFactory.CreateAccount(accountType);
            account.Deposit(1000); // Sample operation to show
account creation.
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine(ex.Message);
        }
        Console.ReadKey();
    }
}

```

This real-time example demonstrates how the Factory Design Pattern encapsulates account creation logic. Suppose the bank introduces new types of accounts in the future (e.g., “Student Account” or “Business Account”). In that case, the system can be easily extended by updating the AccountFactory and adding the respective account classes without altering the existing client code. When you run the above code, you will get the following output.

```

Enter the type of account you'd like to create (savings, current, fixeddeposit):
current
Deposited 1000 to Current Account. New Balance: 1000

```

Real-Time Example of Factory Design Pattern in C#: Report Generation

Imagine you are building a system to generate various reports for an organization – like sales, inventory, and HR reports. Using the Factory Design Pattern, we can instantiate the desired report generator based on a user’s selection. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```

using System;
namespace FactoryDesignPattern
{

```

```

//Define the Product Interface
public interface IReportGenerator
{
    void GenerateReport();
}
//Concrete Implementations for the Products
public class SalesReport : IReportGenerator
{
    public void GenerateReport()
    {
        Console.WriteLine("Generating Sales Report...");
        // Logic for generating sales report
    }
}
public class InventoryReport : IReportGenerator
{
    public void GenerateReport()
    {
        Console.WriteLine("Generating Inventory Report...");
        // Logic for generating inventory report
    }
}
public class HRReport : IReportGenerator
{
    public void GenerateReport()
    {
        Console.WriteLine("Generating HR Report...");
        // Logic for generating HR report
    }
}
//Factory Class to Produce the Products
public static class ReportFactory
{
    public static IReportGenerator CreateReport(string reportType)
    {
        switch (reportType.ToLower())
        {
            case "sales":
                return new SalesReport();
            case "inventory":
                return new InventoryReport();
            case "hr":
                return new HRReport();
            default:
                throw new ArgumentException("Invalid report type");
        }
    }
}

// Testing the Factory Design Pattern
public class Program
{
    public static void Main()
    {

```

```

        Console.WriteLine("Select the type of report you want to
generate (sales, inventory, hr): ");
        string reportType = Console.ReadLine();
        try
        {
            IReportGenerator reportGenerator =
ReportFactory.CreateReport(reportType);
            reportGenerator.GenerateReport();
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine(ex.Message);
        }
        Console.ReadKey();
    }
}
}

```

In this example, the Factory Design Pattern allows the organization to extend its reporting functionalities easily. If a new type of report is introduced in the future (e.g., a “Financial Report”), we need to create the FinancialReport class and extend the ReportFactory without making changes to the client code. When you run the above code, you will get the following output.

```

Select the type of report you want to generate (sales, inventory, hr):
sales
Generating Sales Report...

```

Real-Time Example of Factory Design Pattern in C#: Cloud Storage System

Imagine you’re designing a cloud storage system where users can choose from different storage providers like Amazon S3, Azure Blob Storage, or Google Cloud Storage. Using the Factory Design Pattern, we can instantiate the appropriate storage provider based on user configuration or preferences. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```

using System;
namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface IStorageProvider
    {
        void SaveFile(string fileName, byte[] fileData);
        byte[] RetrieveFile(string fileName);
    }
    //Concrete Implementations for the Products
    public class AmazonS3Provider : IStorageProvider
    {
        public void SaveFile(string fileName, byte[] fileData)
        {
            Console.WriteLine($"Saving {fileName} to Amazon S3...");
            // Logic for saving file to Amazon S3
        }
        public byte[] RetrieveFile(string fileName)
        {
            Console.WriteLine($"Retrieving {fileName} from Amazon S3...");
        }
    }
}

```

```

        // Logic for retrieving file from Amazon S3
        return new byte[0]; // Sample return
    }
}
public class AzureBlobProvider : IStorageProvider
{
    public void SaveFile(string fileName, byte[] fileData)
    {
        Console.WriteLine($"Saving {fileName} to Azure Blob
Storage...");
        // Logic for saving file to Azure Blob Storage
    }
    public byte[] RetrieveFile(string fileName)
    {
        Console.WriteLine($"Retrieving {fileName} from Azure Blob
Storage...");
        // Logic for retrieving file from Azure Blob Storage
        return new byte[0]; // Sample return
    }
}
public class GoogleCloudStorageProvider : IStorageProvider
{
    public void SaveFile(string fileName, byte[] fileData)
    {
        Console.WriteLine($"Saving {fileName} to Google Cloud
Storage...");
        // Logic for saving file to Google Cloud Storage
    }
    public byte[] RetrieveFile(string fileName)
    {
        Console.WriteLine($"Retrieving {fileName} from Google Cloud
Storage...");
        // Logic for retrieving file from Google Cloud Storage
        return new byte[0]; // Sample return
    }
}
//Factory Class to Produce the Products
public static class StorageProviderFactory
{
    public static IStorageProvider CreateStorageProvider(string
providerName)
    {
        switch (providerName.ToLower())
        {
            case "amazons3":
                return new AmazonS3Provider();
            case "azureblob":
                return new AzureBlobProvider();
            case "googlecloud":
                return new GoogleCloudStorageProvider();
            default:
                throw new ArgumentException("Invalid storage provider
name");
        }
    }
}

```

```

    }
}

// Testing the Factory Design Pattern
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Select your storage provider (AmazonS3,
AzureBlob, GoogleCloud): ");
        string providerName = Console.ReadLine();
        try
        {
            IStorageProvider storageProvider =
StorageProviderFactory.CreateStorageProvider(providerName);
            storageProvider.SaveFile("sample.txt", new byte[] { 0x01,
0x02 }); // Example of using the storage provider
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine(ex.Message);
        }
        Console.ReadKey();
    }
}
}

```

Introducing a new storage provider, like Dropbox or another third-party service, becomes very straightforward with this setup. You'd implement a new provider class and extend the StorageProviderFactory without changing the client code. This keeps the code modular and easily extendable, showcasing the power of the Factory Design Pattern. When you run the above code, you will get the following output.

```

Select your storage provider (AmazonS3, AzureBlob, GoogleCloud):
AmazonS3
Saving sample.txt to Amazon S3...

```

Real-Time Example of Factory Design Pattern in C#: UI Theme System

Suppose you're building a software application that supports multiple UI themes, like "Dark Mode," "Light Mode," and "Blue Mode". Users can select their preferred theme from the settings, and the application will apply the chosen theme throughout the UI. The Factory Design Pattern can instantiate the appropriate theme object based on user preferences. Let us see how we can implement the above example using the Factory Design Pattern in C#:

```

using System;
namespace FactoryDesignPattern
{
    //Define the Product Interface
    public interface ITheme
    {
        string BackgroundColor { get; }
        string TextColor { get; }
        void ApplyTheme();
    }
    //Concrete Implementations for the Products

```

```

public class DarkMode : ITheme
{
    public string BackgroundColor => "#000000";
    public string TextColor => "#FFFFFF";
    public void ApplyTheme()
    {
        Console.WriteLine("Applying Dark Mode...");
        // Logic for applying dark theme elements
    }
}

public class LightMode : ITheme
{
    public string BackgroundColor => "#FFFFFF";
    public string TextColor => "#000000";
    public void ApplyTheme()
    {
        Console.WriteLine("Applying Light Mode...");
        // Logic for applying light theme elements
    }
}

public class BlueMode : ITheme
{
    public string BackgroundColor => "#0000FF";
    public string TextColor => "#FFFFFF";
    public void ApplyTheme()
    {
        Console.WriteLine("Applying Blue Mode...");
        // Logic for applying blue theme elements
    }
}

//Factory Class to Produce the Products
public static class ThemeFactory
{
    public static ITheme CreateTheme(string themeName)
    {
        switch (themeName.ToLower())
        {
            case "dark":
                return new DarkMode();
            case "light":
                return new LightMode();
            case "blue":
                return new BlueMode();
            default:
                throw new ArgumentException("Invalid theme specified");
        }
    }
}

// Testing the Factory Design Pattern
public class Program
{
    public static void Main()
    {

```

```

        Console.WriteLine("Select the UI theme (Dark, Light, Blue): ");
        string themeName = Console.ReadLine();
        try
        {
            ITheme theme = ThemeFactory.CreateTheme(themeName);
            theme.ApplyTheme();    // Applying the selected UI theme
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine(ex.Message);
        }
        Console.ReadKey();
    }
}

```

In this UI theme system example, the Factory Design Pattern makes it straightforward to extend theme support. If a new “Red Mode” theme is introduced in the future, the process is as simple as adding a new RedMode class and updating the ThemeFactory accordingly. This ensures that theme creation logic is centralized and decoupled from the main application, promoting modularity and maintainability. When you run the above code, you will get the following output.

```

Select the UI theme (Dark, Light, Blue):
Dark
Applying Dark Mode...

```

Real-Time Applications of Factory Design Patterns in C#

Here are some real-time applications or scenarios where the Factory Design Pattern can be applied:

- **User Interface Control Creation:** In applications with complex user interfaces, such as Windows forms or WPF applications, the Factory pattern can be used to create controls (like buttons, text boxes, or custom components) dynamically based on user actions or configurations.
- **Database Access:** In applications that interact with different databases (like SQL Server, Oracle, or MySQL), a factory can be used to instantiate the appropriate database connection and command objects without hard-coding the database type in the business logic.
- **Logging Frameworks:** For applications that require logging functionalities, a factory can provide appropriate logging objects (like file logger, database logger, or cloud-based logger) based on the application settings or environment.
- **Configuration Management:** When managing various configurations (like development, staging, and production), a factory can create configuration objects specific to the current environment, abstracting the details from the rest of the application.
- **Payment Processing Systems:** In systems that need to process payments through different gateways (like PayPal, Stripe, or credit card), a factory can instantiate the correct payment processing class based on the payment method selected.
- **Game Development:** In game development, a factory pattern can be used to create characters, weapons, or other game elements dynamically based on the game state or user choices.
- **Plugin or Module Systems:** The Factory pattern can be used to instantiate the appropriate plugin based on user actions or configuration files for applications that support plugins or modules.
- **API Integration:** When integrating with various external APIs (like social media, weather services, or geolocation services), a factory can provide the correct API client based on the service required.
- **Object Pooling:** A factory can manage and reuse these objects efficiently when object creation is expensive (like threads or database connections).

Advantages and Disadvantages of Factory Design Pattern in C#

Like all design patterns, the Factory Design Pattern has advantages and disadvantages. Let's explore both aspects:

Advantages of Factory Design Pattern in C#:

- **Loose Coupling:** The Factory Pattern decouples the implementation of the product from its use. This means that the client code doesn't need to know the specifics of how to create the product.
- **Flexibility and Scalability:** Introducing new concrete products without changing the client code is easy. This makes the application more flexible and scalable.
- **Single Responsibility Principle:** The factory class encapsulates the creation logic for the products, adhering to the Single Responsibility Principle. This leads to cleaner, more organized code.
- **Open/Closed Principle:** The Factory Pattern aligns with the Open/Closed Principle as you can introduce new types of products without altering existing factory or client code.
- **Manage Complexity:** It can manage and centralize complex creation logic, especially useful when creating objects requires more than simple instantiation.
- **Control Over Instantiation:** Factories can control how and when objects are created. For instance, you can implement singleton, prototype, or pool patterns within a factory.

Disadvantages of Factory Design Pattern in C#:

- **Complexity:** Introducing a factory pattern can add complexity to the code, especially when a simple object creation would suffice.
- **Indirection:** There's an extra layer of abstraction, which can sometimes make the code harder to understand and debug, especially for those unfamiliar with the pattern.
- **Overhead:** In performance-critical applications, the additional method call and object creation overhead can be a drawback.
- **Dependency on Factories:** The client code still depends on the factory for creating objects, which can lead to problems like the proliferation of factory classes for complex systems.
- **Refactoring Existing Code:** Integrating the Factory Pattern into existing code that wasn't designed with this pattern in mind can be challenging and require significant refactoring.
- **Requires Proper Design:** Misuse or incorrect implementation of the Factory Pattern (like using too many factories or incorrect abstraction) can lead to maintenance issues and increase the difficulty of further modifications.

In the next article, I will discuss the [Factory Method Design Pattern in C#](#) with Examples. Here, in this article, I try to explain **Real-Time Examples of Factory Design Patterns in C#**. I hope you enjoy this Factory Design Pattern Real-Time Examples using C# article.